

docs_chain

Ningún documento rastreado puede desincronizarse — con hash de contenido, bidireccional y atómico.

Resumen

Docs Chain es un motor universal de propagación de dependencias entre documentos y bases de datos, implementado en Go y bidireccional. Cuando cualquier miembro de una cadena registrada cambia —fuente en Markdown, una exportación en HTML/PDF/DOCX o una base de datos SQLite—, detecta el cambio mediante hash de contenido y lo propaga de forma atómica a todos los miembros conectados.

Descripción breve

Un motor Go que mantiene sincronizados documentos y bases de datos. Mediante recomputación incremental con hash de contenido al estilo Salsa sobre un DAG (orden topológico de Kahn, corte temprano, aristas de sincronización bidireccional, confirmaciones atómicas con renombrado + transacciones SQLite), regenera las exportaciones cada vez que cambia algún artefacto vinculado.

Descripción detallada

Docs Chain es lo que se construye después de escribir por enésima vez el mismo frágil script de shell para “regenerar el PDF cuando cambia el Markdown”. Reemplaza todo ese género de sincronización artesanal con un motor real. Modela los documentos y bases de datos de un proyecto como miembros de una cadena y, cuando alguno cambia, propaga ese cambio a todos los miembros conectados en todas las direcciones declaradas, regenerando y reexportando de forma atómica para que ningún artefacto rastreado pueda desincronizarse jamás. El diseño toma su rigor directamente del mundo de los sistemas de compilación incremental, no del scripting: la detección de cambios se basa en **hash de contenido, no en *mtime***, por lo que un touch no activa

nada y una edición de un solo byte desencadena exactamente las recompilaciones que debe —sin falsas alarmas ni cambios omitidos. Formalizado en una sola línea, es recomputación incremental con hash de contenido al estilo Salsa sobre un DAG, con orden topológico de Kahn, corte temprano que poda subárboles sin cambios, aristas de sincronización bidireccional con autoridad declarada, y confirmaciones atómicas mediante renombrado + transacciones SQLite, de modo que un fallo durante la propagación nunca deje una exportación a medio escribir. Se distribuye como submódulo de vasic-digital y se consume como parte central del submódulo HelixConstitution, por lo que cualquier proyecto que adopte la constitución obtiene Docs Chain de serie y registra sus propias cadenas mediante YAML por contexto. La implementación es transparente en cuanto a su estado (según la constitución §11.4.6): las Fases 1-4 (DAG central + hashing, adaptadores/transformaciones de nodos, orquestador de propagación con atomicidad, CLI multicontesto con comandos sync/verify/doctor/graph/watch) están implementadas y probadas; la Fase 4b añade *builtins* genéricos bidireccionales md-to-sqlite/sqlite-to-md (en puro Go, detección de desincronización a nivel de fila, ida y vuelta estable en bytes) y un *builtin* colorize-html; la Fase 5 (pruebas e2e reales con binarios completos) está implementada y en VERDE. Las Fases 6-7 (distribución de la constitución, integración con ATMOSphere) siguen en PLANIFICACIÓN y con acceso restringido a operadores. Herald es el primer consumidor real en producción, sincronizando un corpus de 66 documentos en múltiples formatos que verifica su correcto funcionamiento.

Por qué lo creamos

La documentación, las exportaciones y las bases de datos se desincronizan en cuanto se mantienen a mano o con scripts frágiles. Docs Chain convierte la sincronización en un proceso mecánico, preciso por hash de contenido y atómico, de modo que un cambio en cualquier punto de una cadena actualiza correctamente y de forma segura todo lo que depende de él (tanto aguas abajo como aguas arriba).

Por qué es un cambio de juego

Toma las garantías de corrección, obtenidas con tanto esfuerzo, que los autores de compiladores y sistemas de construcción dan por sentadas —grafos de dependencias con hash de contenido, recomputación mínima, commits atómicos— y las aplica a la documentación y las bases de datos, un ámbito que históricamente ha avanzado a trompicones con tareas programadas y buenas intenciones. La sincronización bidireccional real implica que la relación entre un origen y su

exportación se mantiene en ambos sentidos, de modo que “la documentación está desactualizada” y “la exportación no coincide con el origen” dejan de ser errores recurrentes para convertirse en estados que el motor simplemente no permitirá que existan.

Qué es innovador

- Recomputación incremental basada en hash de contenido (no en *mtime*) sobre un DAG con corte anticipado.
- Sincronización bidireccional con aristas de autoridad declarada (docs ↔ exportaciones ↔ SQLite).
- Commit con renombrado atómico + transacción SQLite para propagación segura ante fallos.
- Viaje de ida y vuelta en Go md-a-sqlite/sqlite-a-md con detección de desviaciones a nivel de fila.

Desafíos y soluciones

- **Reconstrucciones espurias:** resueltas mediante detección por hash de contenido en lugar de marcas de tiempo.
- **Actualizaciones parciales o corruptas:** resueltas con renombrado atómico y transacciones SQLite.
- **Ordenación correcta en miembros múltiples:** resuelta con ordenación topológica de Kahn + corte anticipado.
- **Informe honesto de capacidades:** resuelto marcando cada fase como IMPLEMENTADA o PLANIFICADA según §11.4.6.

Pila tecnológica (por qué y cómo)

- **Go** — motor completo (internal/hash, graph, adapter, orchestrator, config, state, runner, cmd/docs_chain).
- **DAG + ordenación topológica de Kahn** — ordenación de dependencias con corte anticipado.
- **SQLite (modernc puro en Go)** — miembros de la base de datos y commits transaccionales.
- **fsnotify** — demonio watch para propagación en tiempo real.
- **Configuración YAML** — registro de cadenas por contexto.
- **exec: transformaciones** — generación conectable de Markdown→HTML/PDF/DOCX.

Transparencia en la hoja de ruta: Las fases 6-7 (distribución de constitución, integración con ATMOSphere) están PLANIFICADAS / restringidas por operador —no implementadas.